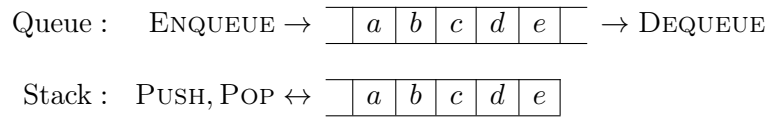
Answers to the mid-term examination on Erlang

Christian Rinderknecht

27 April 2007

A *queue* is a like stack where items are pushed on one end and popped on the other end. Adding an item in a queue is called *enqueuing*, whereas removing one is called *dequeuing*. Compare the following figures:

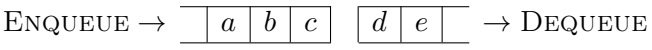


Let `enqueue(E,Q)` be the queue where `E` is the last item in and `Q` is the remaining queue. Let `dequeue(Q)` be the pair `{E,R}` where `E` is the first item out of queue `Q` (if there is none, `dequeue(Q)` is undefined) and `R` is the remaining queue.

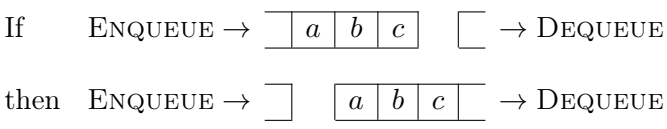
Question. Define `enqueue/2` and `dequeue/1` in the following two cases.

1. **A one-stack implementation.** A simple idea to implement one queue is to use one stack. In this case, Enqueue is simply Push and Dequeue removes the item at the bottom of the stack.

2. **A two-stack implementation.** We can implement one queue with two stacks instead of one: one for enqueuing, one for dequeuing.

   So Enqueue is Push on the first stack and Dequeue is Pop on the second. If the second stack is empty, we swap the stacks and reverse the (new) second:

   Let the pair `{S,T}` denote the queue where `S` is the stack for enqueuing and `T` the stack for dequeuing.

Answer.

1. **A one-stack implementation.**

```
-module(queue1).
-export([enqueue/2,dequeue/1,dequeue_bis/1]).

enqueue(E,Q) -> [E|Q].

% Recursive but not tail-recursive:
dequeue([E])   -> {E,[]};
dequeue([E|Q]) -> {F,R} = dequeue(Q), {F,[E|R]}.

rev(L)       -> rev(L,[]).
rev([],A)    -> A;
rev([H|T],A) -> rev(T,[H|A]).

% Not recursive:
dequeue_bis([H|T]) -> [E|R] = rev([H|T]), {E,rev(R)}.
```

2. **A two-stack implementation.**

```
-module(queue2).
-export([enqueue/2,dequeue/1]).

rev(L)       -> rev(L,[]).
rev([],A)    -> A;
rev([H|T],A) -> rev(T,[H|A]).

enqueue(E,{S,T}) -> {[E|S],T}.

dequeue({S,[E|T]})  -> {E,{S,T}};
dequeue({[H|S],[]}) -> [E|R] = rev([H|S]), {E,{[],R}}.
```

Question. Which of the one-stack or two-stack implementation is the most efficient for dequeuing? What are the best and worst cases for each?

Answer. In the one-stack implementation, every time we dequeue, we must traverse all the items in the stack. In the two-stack implementation, we only traverse the second stack, which is always shorter or equal than the stack in the first case.

1. The best case for the one-stack implementation is when the queue contains only one item, and the best case for the two-stack implementation is when the second stack is not empty: any dequeuing then has a constant cost (a pop).

2. All configurations of the one-stack implementation is the worst case: the cost of dequeuing is always the size of the stack, whilst the worst case for the two-stack implementation is when the second stack is empty, in which case the first stack has to be reversed, so the total cost is the number of items in the queue (which are then all in the first stack).

As a conclusion, the two-stack implementation is more efficient.